

ESP32 Line Follower Robot

Engineering Issues & Resolutions

Overview

This document records the engineering challenges encountered during the development of the ESP32-based line follower robot, together with the diagnostic process and resolution applied in each case. Issues are grouped by domain — electrical, mechanical, software, and tuning — and presented in order of discovery.

The intent is to serve as a reference for future builds and to provide an honest account of the iterative nature of embedded robotics development.

1. Issue Summary

#	Category	Issue	Resolution
1	Electrical	ESP32 resetting on sharp turns	Decoupling capacitor on 5 V rail
2	Electrical	Wires pulling out of screw terminals	Ferrules and heatshrink
3	Mechanical	Caster wheel off-centre causing wobble	Recentred caster mount
4	Mechanical	Weight distribution causing nose dip on stop	Shifted ballast rearward
5	Software	Oscillation on straights (P-only control)	Added derivative (D) term
6	Software	Steering value spikes saturating motors	Low-pass filter on steering output
7	Software	Sensor readings inconsistent across sessions	Boot-time calibration routine
8	Tuning	Robot missing corners at high speed	Reduced base speed
9	Tuning	Frequent reflashing for parameter changes	Live Bluetooth parameter tuning

2. Electrical Issues

Issue #1 · ESP32 Brownout Reset During Sharp Turns

Category: Electrical

Problem The ESP32 would reset mid-run whenever the robot executed a sharp pivot turn. This was intermittent and initially difficult to reproduce consistently.

Root Cause. Sharp pivot turns draw peak current from both motors simultaneously. This transient load caused a voltage dip on the shared 5 V supply rail sufficient to trigger the ESP32's internal brownout detector, initiating a reset.

Resolution A 1000 μ F electrolytic capacitor was added across the 5 V rail close to the ESP32's power input pins. This acts as a local charge reservoir, absorbing the transient and preventing the rail from collapsing during high-current motor events.

Outcome. Resets ceased entirely. The robot now completes sharp pivot turns reliably without any microcontroller interruption.

Issue #2 · Wires Pulling Out of Screw Terminals

Category: Electrical

Problem Motor and power wires would periodically lose contact with the screw terminals on the motor driver, causing intermittent power loss or erratic motor behaviour during operation.

Root Cause. Stranded wire under a screw terminal is prone to individual strand splaying and cold-flow under the screw head. Vibration from the motors and chassis accelerated loosening over time.

Resolution All stranded wire ends were terminated with bootlace ferrules crimped using a ratchet crimping tool. Heatshrink was applied over each ferrule and the adjacent wire jacket to provide strain relief. Terminals were re-torqued after the first test run.

Outcome. Connections became fully reliable. No further intermittent contact failures were observed across subsequent test sessions.

3. Mechanical Issues

Issue #3 · Caster Wheel Off-Centre Causing High-Speed Wobble

Category: Mechanical

Problem At elevated speeds the rear of the robot exhibited a lateral oscillation (shimmy), making straight-line tracking unstable and increasing the difficulty of PID tuning.

Root Cause. The caster wheel assembly had been mounted slightly off the robot's longitudinal centreline during initial assembly. This asymmetry introduced a net lateral moment under forward motion, which amplified into an oscillating shimmy at speed.

Resolution The caster mount was repositioned to the geometric centreline of the chassis and re-fastened. Alignment was verified using a straightedge along the chassis before and after the fix.

Outcome. High-speed wobble was eliminated. Straight-line stability improved noticeably, and subsequent PID tuning became significantly more consistent.

Issue #4 · Weight Distribution Causing Nose Pitch on Deceleration

Category: Mechanical

Problem When the robot decelerated sharply — particularly after a corner recovery — the front of the chassis would pitch downward momentarily, causing the sensor array to dip toward the track surface and produce spurious readings.

Root Cause. The majority of the robot's mass (battery, electronics) was concentrated toward the front of the chassis. Under deceleration the inertial load shifted further forward, causing the nose to drop relative to the caster-supported rear.

Resolution The LiPo battery was relocated rearward on the chassis, redistributing mass toward the centre of the wheelbase. This reduced the forward pitch moment under deceleration.

Outcome. Nose pitch on stopping was eliminated. Sensor readings remained stable through deceleration events and false corner detections caused by sensor dipping ceased.

4. Software Issues

Issue #5 · Sustained Oscillation on Straights with Proportional-Only Control

Category: Software / Control

Problem With only a proportional gain term active, the robot exhibited continuous side-to-side oscillation even on straight sections of track. Reducing K_p reduced oscillation but made cornering sluggish; no value of K_p alone produced stable straight-line tracking.

Root Cause. Proportional-only control produces a correction output proportional to the current error, but has no mechanism to anticipate or damp the overshoot that results from its own correction. The robot continuously overcorrects in alternating directions, producing sustained oscillation.

Resolution A derivative term (K_d) was introduced into the steering equation, producing a PD controller. The derivative term generates a correction proportional to the *rate of change* of error, opposing rapid changes and damping the overshoot inherent in proportional-only control.

Outcome. Oscillation on straights was resolved. The robot now tracks the line smoothly at speed, with $K_d = 1.500$ providing adequate damping without sluggish cornering response.

Issue #6 · Steering Value Spikes Saturating Motor Outputs

Category: Software / Signal Processing

Problem Occasionally the computed steering correction would spike to a large value for one or two control loop cycles, causing the motors to briefly saturate at maximum output. This produced a sudden jerk in the robot's trajectory.

Root Cause. The derivative term amplifies high-frequency noise in the error signal. Transient noise on the QTR sensor readings — caused by surface irregularities or electrical interference — produced large instantaneous changes in error, which the derivative term then amplified into correction spikes.

Resolution A first-order low-pass filter was applied to the steering output in firmware, characterised by a filter coefficient $\alpha = 0.40$. The filtered output is computed as:

$$y_n = \alpha \cdot u_n + (1 - \alpha) \cdot y_{n-1}$$

where u_n is the raw correction and y_{n-1} is the previous filtered value. This smooths transient spikes while preserving adequate bandwidth for legitimate steering corrections.

Outcome. Steering spikes were eliminated. Motor response became smooth and consistent, with no further saturation events observed during normal operation.

Issue #7 · Inconsistent Sensor Readings Across Sessions

Category: Software / Sensing

Problem The QTR-8RC sensor array produced different absolute reflectance readings for the same track surface between power cycles, particularly when ambient lighting conditions changed. This required manual adjustment of detection thresholds between sessions.

Root Cause. The QTR-8RC sensors report a charge-time value that is influenced by ambient light level, surface properties, and sensor-to-ground height. These conditions vary between sessions, shifting the absolute output values even for the same physical track.

Resolution A calibration routine was implemented that executes automatically at power-on. The robot slowly sweeps the sensor array across the track surface for a fixed duration, during which the firmware records the minimum and maximum reflectance value observed by each sensor channel. These per-channel extremes are used to normalise all subsequent readings to a consistent 0–1000 range, making the control algorithm independent of absolute sensor output.

Outcome. Sensor readings became consistent and repeatable across sessions regardless of ambient lighting or minor variation in sensor height. Detection thresholds no longer required manual adjustment between runs.

5. Tuning Issues

Issue #8 · Robot Failing to Detect Corners at High Speed

Category: Tuning

Problem At higher base speeds the robot would frequently overshoot corners, continuing straight through a turn rather than initiating the pivot recovery sequence.

Root Cause. Corner detection requires a minimum number of consecutive sensor readings (`cornerCount` = 5) to exceed the detection threshold before a corner is confirmed. At high speed, the robot traverses the corner detection zone in fewer control loop cycles than the debounce count requires, causing the condition to never be confirmed.

Resolution Base speed was reduced from an initial value to `baseSpeed` = 55, slowing the robot sufficiently for the corner detection debounce to accumulate the required consecutive readings before the corner is passed. This represents a deliberate trade-off between straight-line speed and corner detection reliability.

Outcome. Corner detection became consistent. The pivot recovery sequence now triggers reliably on all corners at the tuned base speed.

Issue #9 · Excessive Iteration Time Due to Reflashing for Parameter Changes

Category: Tuning / Development Workflow

Problem Adjusting PID gains or motion parameters required modifying the firmware source, recompiling, and flashing the ESP32 via USB for each change. With the robot mounted on the chassis this was physically inconvenient and significantly slowed the tuning process.

Root Cause. No runtime interface existed for modifying parameters without a firmware update. All tuning values were compiled as constants.

Resolution A live Bluetooth tuning interface was implemented using the ESP32's integrated Bluetooth stack. A simple serial protocol allows parameter values (K_p , K_d , `baseSpeed`, etc.) to be updated at runtime from a paired device without interrupting operation. Updated values take effect on the next control loop cycle and can optionally be persisted to flash memory.

Outcome. Tuning iteration time was dramatically reduced. Parameter changes can now be applied and evaluated within seconds, accelerating the convergence of the PID gains to their final tuned values.

Conclusion

The nine issues documented above span the full stack of embedded robotics development — from power supply integrity and mechanical alignment through to control algorithm design and development workflow. Several problems that initially appeared unrelated (brownout resets, sensor inconsistency, steering spikes) shared a common underlying theme: insufficient isolation between subsystems, whether electrical, mechanical, or in the signal domain.

Each resolution contributed meaningfully to the final reliability and performance of the platform, and the iterative process of diagnosing and fixing these issues produced a more robust understanding of the system as a whole.